1 Rule-based normalisations with a two-level merge

The solution has six steps.

1. Parse each report into the final report format.
2. Normalise the URL so equivalent addresses compare equal: lower-case the scheme and host, remove default ports and trailing slashes, and compare query strings in a consistent way (for example by parameter name).
3. Map each finding to a weakness class using a small rules table, so that different wordings from different scanners end up in the same class.
4. Level 1 (merge): Findings with the same weakness class and the same normalised location become one record.
5. Level 2 (link): Findings with the same class but a different location stays as separate records and are linked as related.
6. Keep every scanner's original detail (title, description, severity, evidence, scanner ID) inside the merged record, so nothing is lost and it stays clear which scanner reported what.

GitLab ignores CWE and WASC identifiers when deduplicating, because they classify groups of vulnerabilities and using them would wrongly treat many different vulnerabilities as duplicates (GitLab Docs). For dynamic scanning, GitLab defines a vulnerability's location as the URL path, HTTP method and HTTP parameters (GitLab Docs).

My normalised location starts with the URL and can be extended with the parameter where both scanners report one.

Example rules (illustrative; the real rules table is built from the scanners' own wording):

- Missing anti-clickjacking protection: the title or description contains "x-frame-options" or "anti-clickjacking".
- Missing content-type sniffing protection: the title or description contains "x-content-type-options".
- Cookie without HttpOnly flag: the title or description contains "httponly".
- Example of how the merge behaves (illustrative; the host name is made up):
- Nikto reports http://dvwa.test:80/login.php, class: missing anti-clickjacking protection. Result: Level 1, merged with the ZAP finding below.
- ZAP reports http://dvwa.test/login.php/, class: missing anti-clickjacking protection. Result: one record listing Nikto and ZAP as sources.
- ZAP reports http://dvwa.test/index.php, class: missing anti-clickjacking protection. Result: Level 2, a separate record linked to the merged one.
- Nikto words the finding as a missing X-Frame-Options header and ZAP words it as a missing anti-clickjacking header (paraphrased). Both match the first rule above. The first two URLs normalise to dvwa.test/login.php, so they merge. The third is the same class at a different location, so it is only linked.

2 More Approaches

2.1 Approach A: Rule-based normalisation with two-level merge. (Simple)

GitLab Docs say that for dynamic scans, duplicates share the same scan type, location and an identifier, and that CWE and WASC are ignored.

- Strengths: It is transparent, keeps each scanner's detail, and the rules table handles different wording.
- Weaknesses: The rules need maintaining and a loose rule can over-merge.

2.2 Approach B: Fingerprint or hash matching. (Simple)

DefectDojo Docs say the algorithm is chosen per scanner parser (scanner-provided ID, hash of chosen fields, or the combined "Unique ID From Tool or Hash Code" option), and the hashed fields are configurable.

- Strengths: Cheap to implement and fast.
- Weaknesses: It only matches when the chosen fields are identical. Nikto and ZAP word titles differently, so a hash that includes the title will not match across them. Tools also differ on which fields to hash: GitLab ignores CWE, while some DefectDojo example settings include it.

2.3 Approach C: Text similarity. (Complex)

Runeson et al. (2007) found in a case study at Sony Ericsson that about two thirds of duplicate defect reports could possibly be found with natural language processing techniques, and different variants gave only minor differences. Zhang et al. (2022) describe that method as converting report text to vectors and comparing them with cosine, Jaccard or Dice similarity.

- Strengths: It can catch wording that my rules do not cover.
- Weaknesses: A similarity threshold must be chosen and tested, and it struggles when different failures share one underlying fault (Zhang et al., 2022).

2.4 Approach D: Machine learning or deep learning. (Complex)

Zhang et al. (2022) found that on a benchmark of recent reports, a simpler retrieval-based method (REP, from 2011) beat newer deep-learning models on most projects. A deep-learning model matched or beat it only on the two largest projects, and the authors suggest deep learning favours more training data.

- Strengths: It can learn patterns without hand-written rules.
- Weaknesses: It needs labelled training data and more effort and is a poor fit for the small number of findings in this project.

3 Preferred Choice

Use the rule-based approach first. It is the quickest to implement, it is transparent (every merge can be traced to a rule), and it keeps which scanner reported what.

Limitations:

- Rules need maintaining as scanner wording changes, and a loose rule can merge findings that are really different.
- The peer-reviewed papers I found are about bug reports, so they support the general ideas but were not tested on Nikto or ZAP output. For scanner findings, the guidance comes from tool documentation (DefectDojo and GitLab), which describes how those tools work rather than measuring how well they work.
- The results come from DVWA, a deliberately vulnerable test application, and only two scanners, so they may not carry over to other targets.

4 Bibliography

1. DefectDojo. (n.d.). *Deduplication tuning (open source)*. DefectDojo Documentation. Retrieved October 7, 2026, from https://docs.defectdojo.com/triage_findings/finding_deduplication/os__deduplication_tuning/
2. GitLab. (n.d.). *Vulnerability deduplication process*. GitLab Docs. Retrieved October 7, 2026, from https://docs.gitlab.com/user/application_security/detect/vulnerability_deduplication/
3. Runeson, P., Alexandersson, M., & Nyholm, O. (2007). Detection of duplicate defect reports using natural language processing. In *Proceedings of the 29th International Conference on Software Engineering (ICSE 2007)* (pp. 499–508). IEEE. https://doi.org/10.1109/ICSE.2007.32
4. Zhang, T., Han, D., Vinayakarao, V., Irsan, I. C., Xu, B., Thung, F., Lo, D., & Jiang, L. (2022). Duplicate bug report detection: How far are we? *arXiv*. https://doi.org/10.48550/arXiv.2212.00548